NeuroSYNC

AI agents for research: simulations, experimental automation, and visualisation

Egor Manuylovich
Aston Institute of Photonic Technologies
Aston University

What this talk is about

NeuroSYNC

01	Introduction to coding agents
02	Scientific simulations
03	Experimental automation
04	Scientific visualisation

What are AI coding agents?

NeuroSYNC

Type	What it does	Example
Chatbot	Answers questions and generates code snippets on demand.	ChatGPT, Claude, Gemini…
Autocomplete	Suggests code while typing, one line or block at a time.	GitHub Copilot, Cursor
Agent	Reads files, edits codebases, runs tests, sees errors, iterates.	Claude Code, Codex, Gemini, Cursor

Key thing: agents can inspect files, modify codebases, write/execute tests, and iterate autonomously.

AI-assisted simulations: what changes?

NeuroSYNC

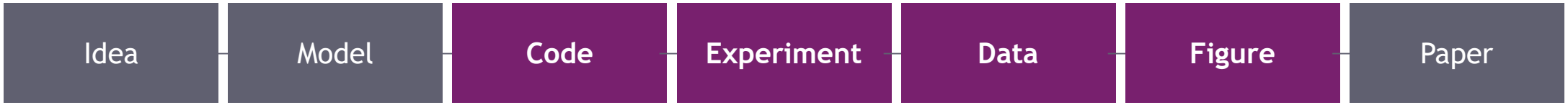
Before	After
• Slow, manual simulation setup	• Parameter sweeps generated quickly
• Fragmented scripts per use-case	• Plotting pipelines automated end-to-end
• Manual plotting and formatting	• Tests and docs added early in the cycle
• Limited parameter-space exploration	• Faster hypothesis iteration
• Documentation as an afterthought	• Modular, reusable simulation components

Examples:

Adopting OOP style and feeling its benefits
Mapping photonic devices to NN layers
Procedural plots
Mode decomposition with orthogonality constraints
Optical interferometer phase simulator with analytical benchmarks

(my own coding skills improved so much after I started using LLMs for coding!)

The research bottleneck is shifting



Highlighted stages: the software bottlenecks where agents can help most.

Research is increasingly software-heavy

Researchers spend significant time on glue code, plotting, automation, UI, debugging, and data wrangling — not just science.

AI agents reduce software friction

Agents can generate, test, refactor, and document software far faster than writing it from scratch.

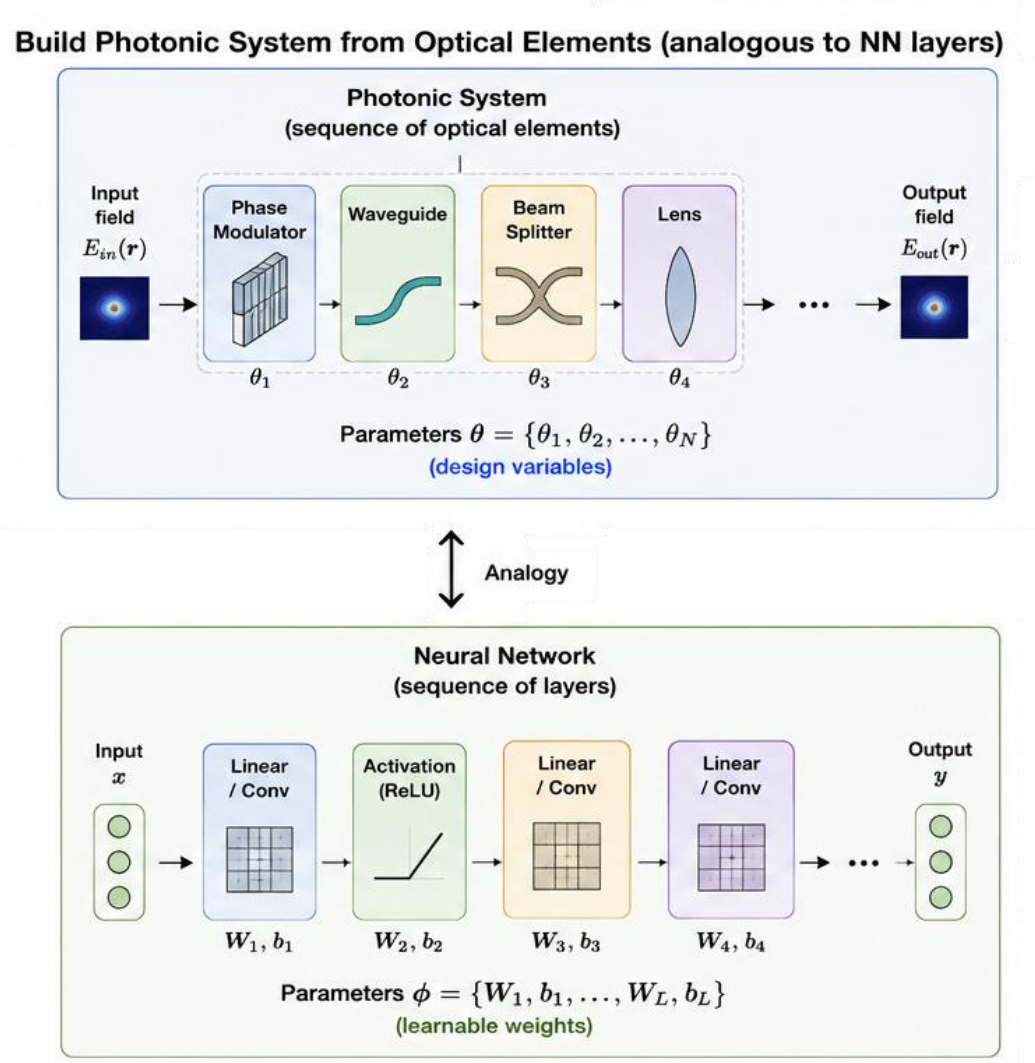
You can now develop things which would take too long to be feasible

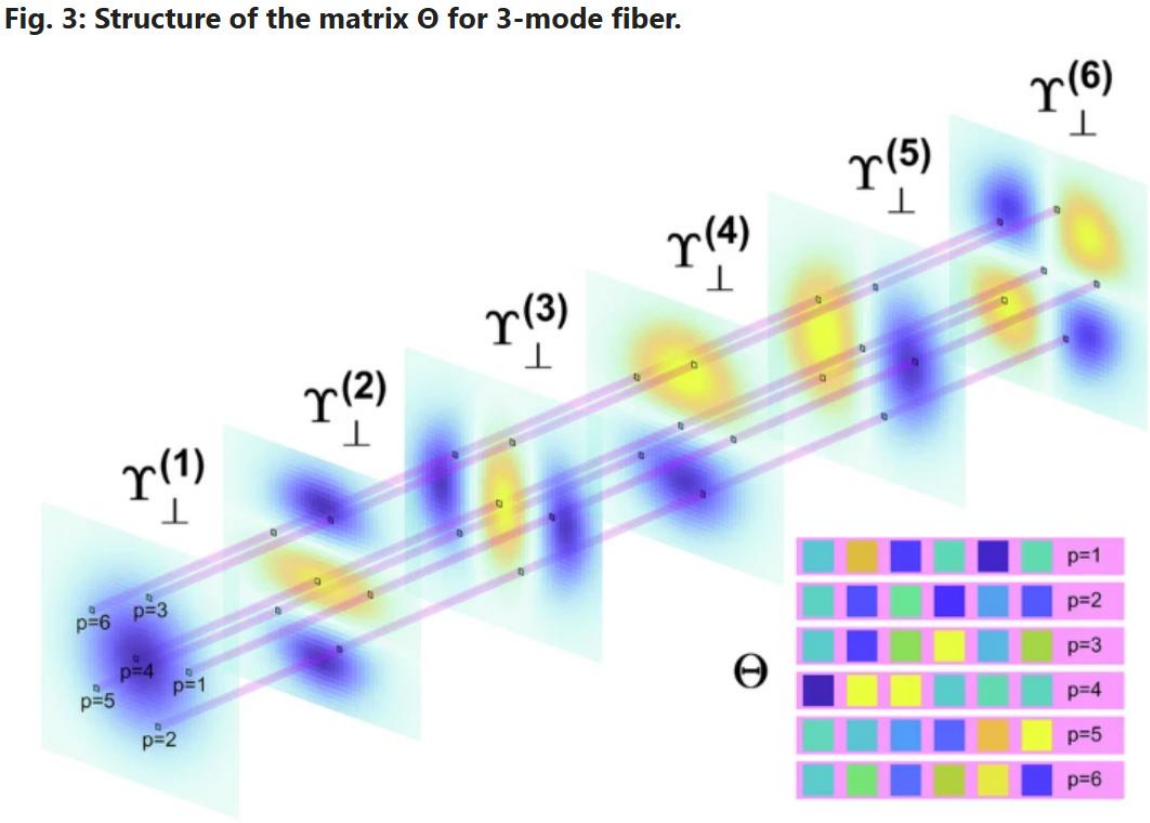
From "writing code" to "specifying and validating systems."

> “Computer simulation is the third branch of science, complementing theory and experiment.”
>
> Robert E. Caflisch, *“Monte Carlo and quasi-Monte Carlo methods”*, Acta Numerica, 1998.

Many things are easy to verify and much harder to solve.

Examples





Many things are easy to verify and much harder to solve.

Vibe coding

NeuroSYNC

Fast prompting with minimal planning — "make me a simulator" — and iterating until it looks right.

Good for	Risk
One-off scripts and quick plots	• Hard to maintain
Educational demos and toy simulations	• Hidden bugs that look fine
Small GUIs and data explorers	• Progressively harder to modify
Fast hypothesis sketching	• Poor reproducibility (same prompt -> different results)
	• Difficult to extend or validate

DEMO 0

Simple optical simulation from a single prompt
https://chatgpt.com/c/6a08dde5-6b70-83eb-8f60-b04852f85e8c

What agents are good at — and not

NeuroSYNC

Good at	Weak at
• APIs, wrappers, boilerplate/templates	Scientific correctness
• Plotting and interactive dashboards	Hidden physical assumptions
• File handling and data pipelines	Numerical validity and stability
• Refactoring and documentation	Physical intuition
• Connecting vendor SDKs	Domain-specific conventions
• Writing and running tests	Reproducibility (without constraints)

The researcher remains the scientific authority (i.e. is responsible for the result!)

Vibe coding -> spec-driven development

NeuroSYNC

Treat the AI agent as a junior research software engineer.

Requirements → Equations / Physics → Architecture → Tests → Implementation → Validation

agent.md	Project instruction file: physics, architecture, tests, conventions.
Acceptance tests	Define what correct output looks like before you start coding (e.g. check energy conservation, edge cases, etc.)
Project structure	Modular design allows agents to work on isolated components.
Reproducibility	Specs make it possible to re-run, audit, and extend the project.

Why specs and tests matter in science

NeuroSYNC

In research, "looks correct" is not sufficient.

Physical assumptions

Steady-state vs dynamic assumptions must be explicit; Orthogonality from

Interferometer phase

Sign convention for reflection vs transmission matters.

Propagation operator

Phase accumulation direction determines physical meaning: Raman shift is to longer wavelengths

Energy conservation

Orthogonality constraint must be built into the implementation.

Human-agent research loop

NeuroSYNC

Human	Agent
Defines the physics and mathematics	Creates project structure
Writes the specification (agent.md/spec.md)	Implements modules from the spec
Sets acceptance criteria and tests	Writes tests and runs sweeps
Validates outputs against theory/edge cases	Refactors and documents code
Checks physical meaning of results	Builds plots and dashboards

You do the science, and agent is your hard-working buddy. You check each step and specify the tests!

Iteration is useful only if each cycle includes explicit validation

Example simulation workflow

NeuroSYNC

Stage 1 Physics specification	Stage 2 Implementation & tests	Stage 3 Plots & interpretation
Define governing equations	Agent creates module structure	Agent generates parameter sweeps
Physical assumptions (State gain saturation model/steady state/dynamical)	Implements operators from spec	Plots transfer functions
Specify conventions (FFT sign, phase conventions)	Writes unit tests for each module	Human interprets results
List known edge cases to test against	Writes physical tests (energy/momentum conservation, photon number conservation…)	Revise spec if needed
	Runs limiting-case checks	

Debugging with agents

NeuroSYNC

Typical subtle bugs in AI-generated scientific software:

Wrong beamsplitter convention	Reflection and transmission phases differ by π which is easy to miss.
Arm-length sum vs. difference	The phase depends on the difference, not the sum of arm lengths.
Propagation sign error	Sign convention in e^{ikz} changes whether wave converges or diverges.
Numerical instability	Can be hidden by plotting and only revealed by edge case tests.
Unit mismatch	Mixing radians and degrees, or nm and μm, in intermediate variables.

Agents speed up debugging and testing, but they also generate subtle scientific bugs.

Testing scientific software

NeuroSYNC

Test types	Science-specific examples
• Unit tests per module	Zero nonlinearity → linear propagation
• Physical limiting cases/edge cases	Zero length → identity transfer matrix
• Conservation law checks	Lossless system → energy conservation
• Symmetry and reciprocity tests	Symmetric beamsplitter → equal splitting
• Analytical benchmark comparisons	Phase wrap at 2π → field continuity
• Parameter sensitivity scans	Degenerate modes → orthogonal eigenfunctions

"If you cannot define a test, you probably do not yet understand the system."

NeuroSYNC

LIVE DEMO 1

AI-assisted simulation: Optical Circuit Lab

The traditional lab software problem

NeuroSYNC

Researchers often juggle separate software for every instrument:

Laser	Oscilloscope	SLM
Camera	Motion stage	Power meter

Pain points:

Manual clicking with no scripting

Poor reproducibility between sessions

No synchronisation between devices

Every new experiment needs custom tooling

AI-generated lab orchestration software

NeuroSYNC

Modern instruments expose programmable interfaces:

SCPI	VISA	Serial / RS-232
Python SDKs	REST APIs	USB HID

How agents help:

Read vendor examples and docs and write device wrappers automatically

Connect multiple device wrappers into a unified Python backend

Generate a dashboard (FastAPI + PyQt / Streamlit) in a single session

Write automated experiment sequences with logging and error handling

From orchestration to closed-loop experiments

(closed loop: output of analysis feeds back into instrument control)

Closed-loop possibilities:

Phase stabilisation and active correction

Adaptive measurement: focus where variance is highest

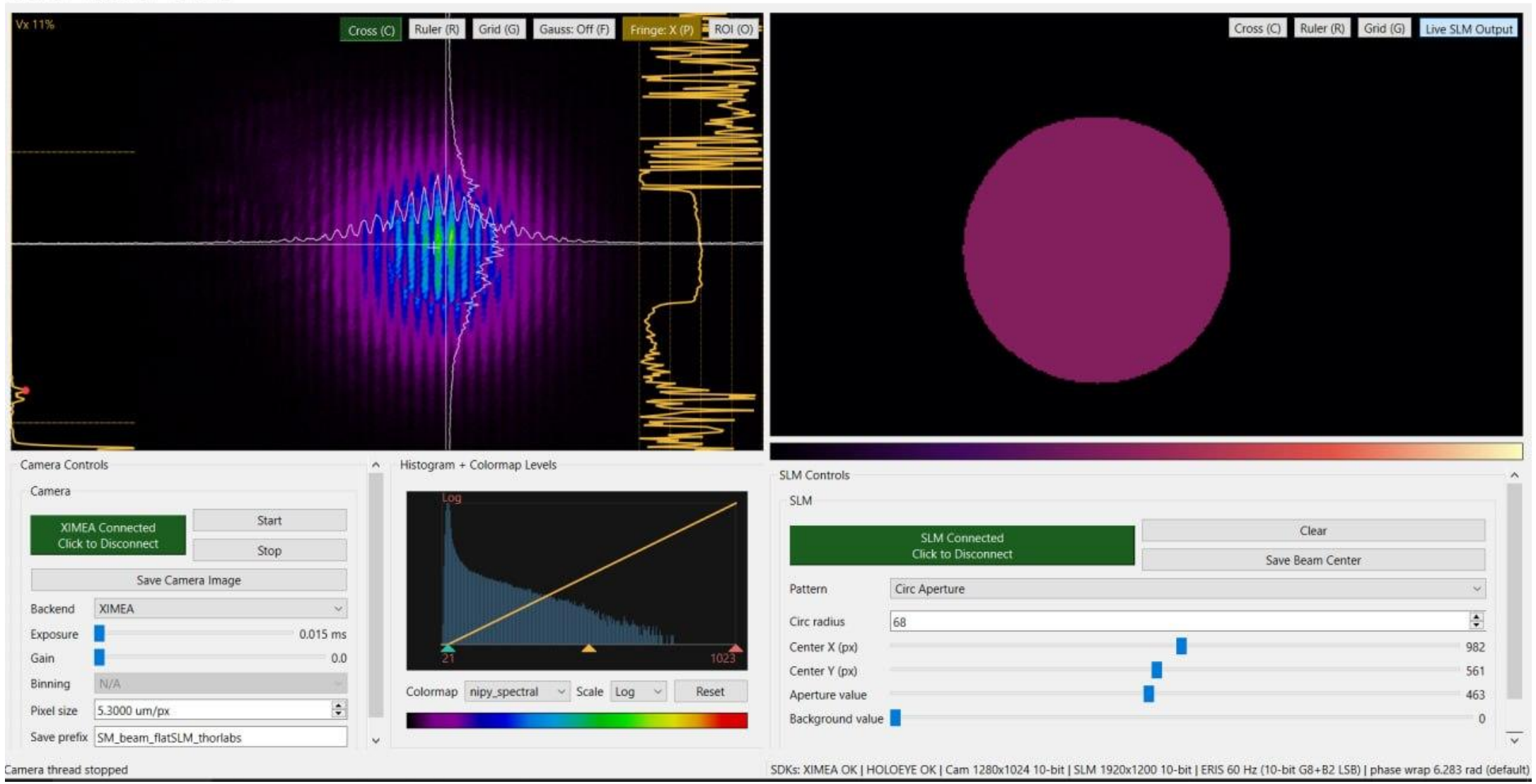
Bayesian optimisation of experimental parameters

Autonomous photonics experiments with minimal human intervention

Reinforcement learning for alignment or mode matching

Example: SLM and camera control software

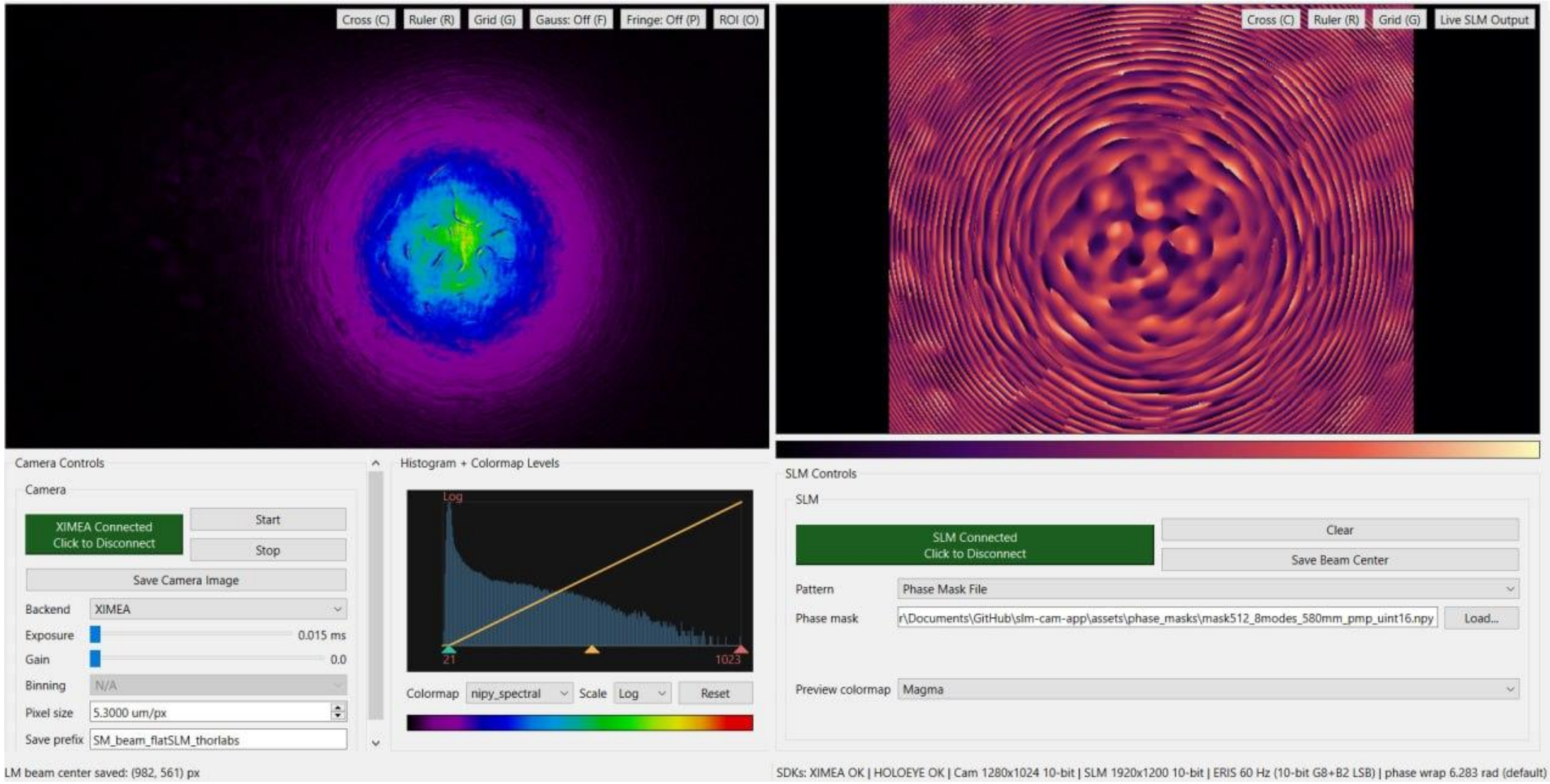
NeuroSYNC



Custom software for SLM-camera calibration and automatic dataset acquisition | Callouts: live camera · SLM pattern

Real example: phase masks and calibration

NeuroSYNC



From manual alignment to reproducible dataset generation | Callouts: measured optical field · generated phase mask

NeuroSYNC

LIVE DEMO 2

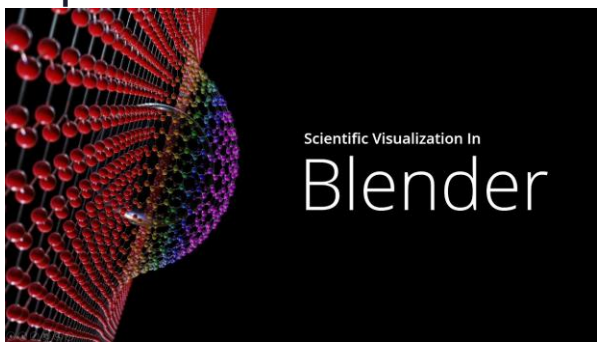
Custom SLM-camera control and lab orchestration software

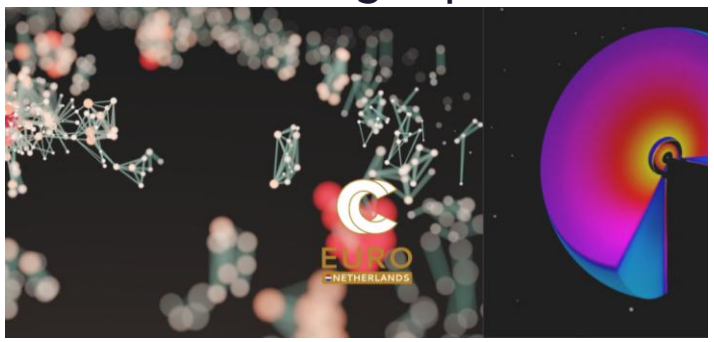
Scientific visualisation matters

NeuroSYNC

Publication-quality figures require(d) a completely different skill: 3D modelling

Top-journal figures must	The problem
Communicate the physical mechanism	Blender is perfect but the learning curve is steep
Guide the reader's intuition	Photorealistic ≠ scientifically correct
Look **nice**, deliberate and unambiguous	Revisions require re-doing geometry
Be reproducible from the source files	Researchers often lack 3D modelling expertise

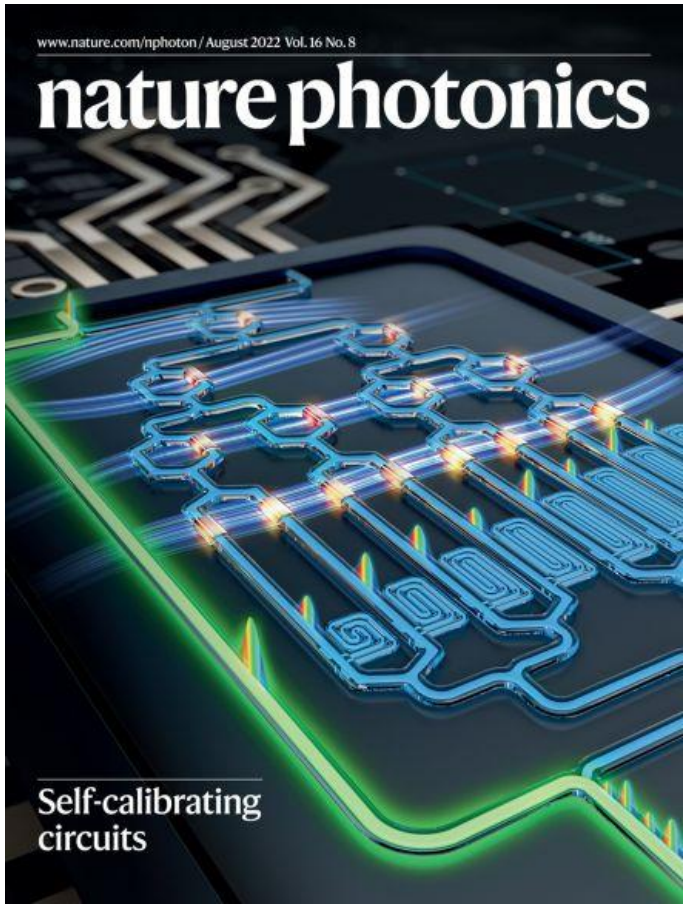


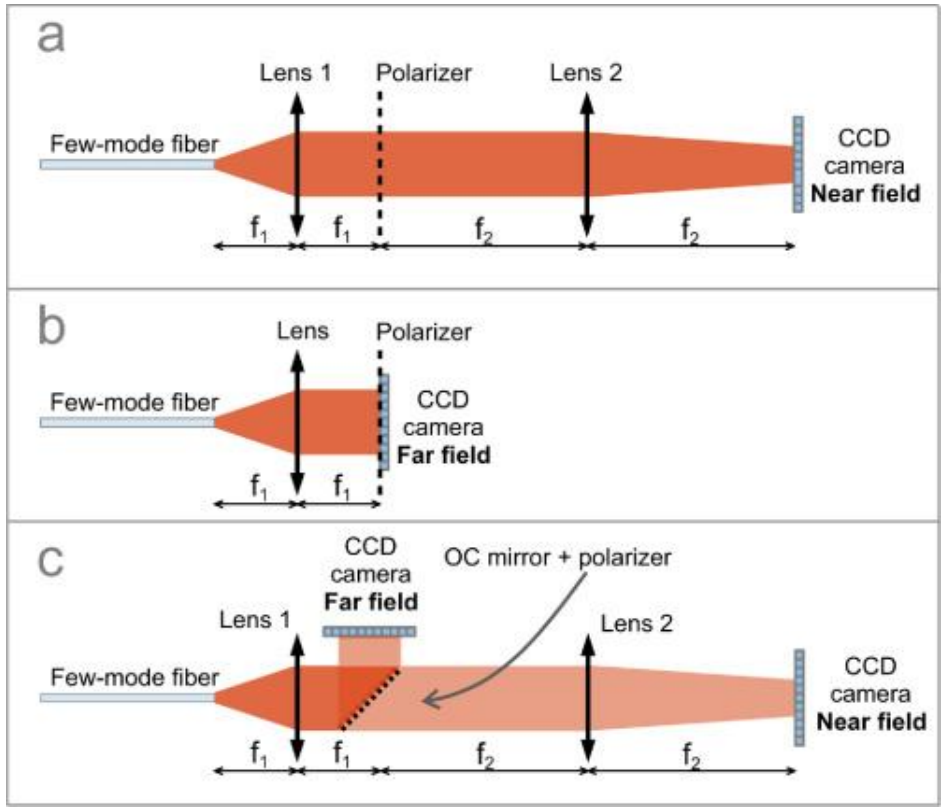


Solution: AI agents controlling Blender via the Model Context Protocol (MCP).

The good, the bad and the ugly

NeuroSYNC





Good **Bad** **Ugly**

Blender MCP + agents

NeuroSYNC

Text specification	Agent (Codex/Claude/...)	Blender MCP (Python API)	Scene + Render	Final Figure

What the agent controls in Blender:

Geometry Meshes, curves, arrays, boolean operations: build yours using a text description.

Materials PBR shaders, emission, glass, subsurface: all physically based by default.

Lighting HDRI environments, area lights, point lights

Camera Field of view, depth of field, focal plane. Easier to set yourself: choose one you like!

Animation Keyframes, physics sims, camera paths: for figures and supplementary material

Blender MCP

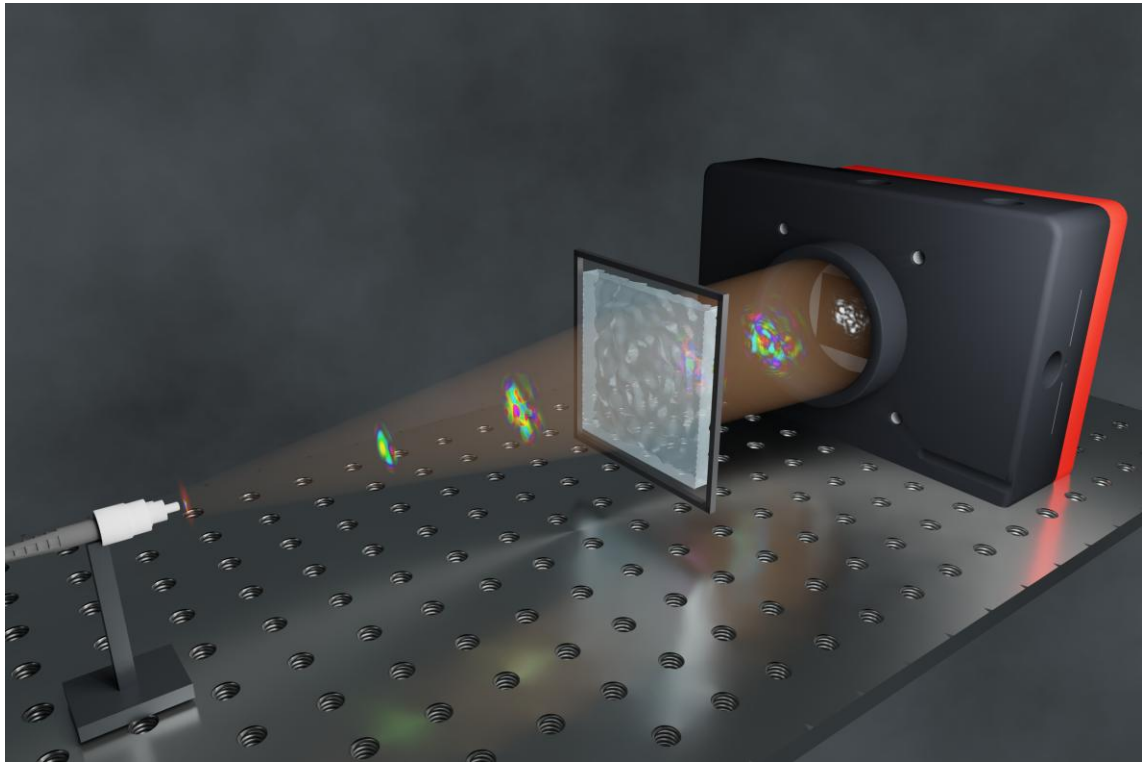
Example visualisation pipeline

NeuroSYNC

1 Describe physical scene	2 Generate geometry	3 Check scientific correctness	4 Render and refine
You write a clear text spec: geometry, scale, material, camera angle.	Agent writes Blender Python script, creates objects, applies materials.	YOU verify the geometry, proportions, and physical conventions.	Iterate on lighting and camera; export final PNG / animation.

Example from my work:

- Multimode fibre propagation: field intensity visualised in 3D
- Phase-mask geometry: actual SLM pixel structure is rendered
- Optical table system: full bench layout for a methods figure
- Speckle-to-speckle transformation: real wavefront evolution



You can do so much more, including scripted 3D animations, guided by real physics, just put the right data to the agent and verify the result.

Practical advice for PhD students

NeuroSYNC

Six habits that separate rigorous agent use from vibe coding:

1. **Learn basic Python and Git first**
   You need to read and review agent output. You cannot do that without basics.
2. **Start with small, well-defined tools**
   A single-device wrapper or a plotting script is the right first project.
3. **Write agent.md files**
   Write the spec before prompting. It forces you to think about the problem (you can use LLM but DO validate it)
4. **Always ask for tests, not just code**
   "Write tests for the edge cases you just specified" is a powerful prompt.
5. **Validate output. Always!**
   Try to break the output. Compare with known limits or analytical results (same as for your own code).
6. **Keep reproducible records**
   Always use git, log the spec, the agent session, and the validation results together.

Limits and pitfalls

NeuroSYNC

Plausible wrongness is more dangerous than obvious failure.

Risks	Mitigation
• Plausible but incorrect code	• Write specs before coding
• Hidden physical assumptions	• Define tests that can fail
• Geometrically convincing but wrong figures	• Validate against analytical limits
• Weak reproducibility without version control	• Commit everything to version control
• Over-trust in demo outputs	• Human reviews every output
• Spec-less "vibe" coding for production use	• Treat agent output as a first draft

Recommended stack

NeuroSYNC

A practical starting point — not the only path:

Coding foundation

Python
VS Code / JupyterLab
Git + GitHub
NumPy · SciPy · Matplotlib

AI agents

Claude Code (CLI)
Cursor (IDE)
GitHub Copilot
OpenAI Codex CLI

Apps & automation

FastAPI + PyQt / Streamlit
Docker
NumPy pipelines

Visualisation

Blender + Blender MCP
Matplotlib / Plotly
Napari (microscopy)

The future researcher skillset: takeaways

NeuroSYNC

AI does not remove the need for expertise, it raises its value.
Agents amplify whatever skill level the researcher brings.

Domain expertise

Deep knowledge of your field is non-negotiable — agents cannot substitute it.

Specification thinking

Ability to describe precisely what a system should do, at the level of equations and tests.

Validation mindset

Healthy scepticism toward any output. Ask "how would I know if this is wrong?".

Software architecture

Basic understanding of modular design, interfaces, and testability.

AI collaboration skills

Knowing what to delegate, how to check, and when to take back control.